

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KHOA HỌC MÁY TÍNH



SELF-STUDY

Đề tài: TRANSFORMERS

Mentor: My self
Student: Lại Khánh Hoàng

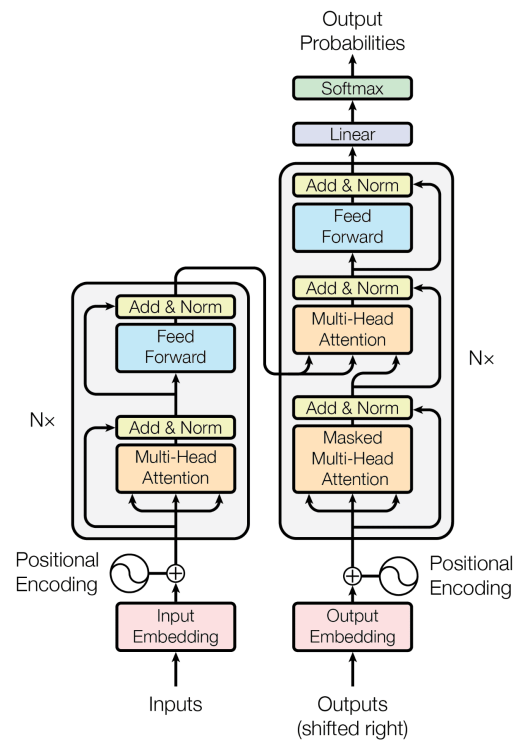
Tp. Hồ Chí Minh



Mục lục

1	Transformers architectures	2
2	Encoder block	3
2.1	Input Embeddings	3
2.2	Positional encoding	3
2.3	Multi-Head Attention	3
2.4	Residual Learning	4
2.5	Feed Forward	4
3	Decoder block	4
3.1	Masked Multi-head Attention: Subsequent mask	4
3.2	Multi-head attention (trong Decoder)	4
4	Training	5
4.1	Loss	5
4.2	Dropout và Learning rate Warmup	6

1 Transformers architectures



Ba điểm chính của Transformers:

- Không tuần tự như RNNs, tất cả input đều được đưa vào cùng một lần qua model và tính toán cùng lúc
- Attention được sinh ra từ chính input của model (Self-attention)
- Nhiều hơn một Attention được sinh ra mỗi lần (Multi-head Attention). Điều này mang lại cho mô hình lợi thế trong việc tập trung vào các từ khác nhau theo nhiều cách khác nhau (h là số lượng đầu attention). Nó mở rộng khả năng của mô hình trong việc chú ý đến các vị trí khác nhau và cung cấp cho lớp attention nhiều biểu diễn khác nhau (chúng ta sẽ có bộ Query, Key và Value khác nhau).



2 Encoder block

2.1 Input Embeddings

Chúng ta thiết lập max_length và pad các câu có độ dài dưới max_length . Ta cũng không muốn model tham gia các pad token, do đó chúng ta phải tạo **mask** cho mỗi câu.

Nếu input data là một ma trận có shape $(batch_size, max_length)$ thì mask cũng là một ma trận có shape $(batch_size, max_length)$. Trong đó 1 nếu chứa từ và 0 nếu là pad token.

Sau đó embed tất cả các từ trong câu, khi đó input data lúc này có shape $(batch_size, max_length, embedding_dim)$.

2.2 Positional encoding

Trong Transformer, mô hình không có tuần tự (order) như RNN, nên ta phải thêm thông tin vị trí cho từng token trong chuỗi đầu vào. Phương pháp phổ biến nhất là Sinusoidal Positional Encoding — tức là thêm một vector “mang thông tin vị trí” vào embedding của mỗi token.

Công thức:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right),$$

$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$

Trong đó:

- pos = vị trí của token (0, 1, 2, ...)
- i = chỉ số của chiều trong embedding vector
- d_{model} = kích thước của embedding (ví dụ 512)

Sau đó cộng thẳng vào embed vector.

2.3 Multi-Head Attention

- Giả sử ta có ma trận đầu vào $X \in \mathbb{R}^{(B, L, d_{model})}$

Trong đó: B : batch size, L : độ dài của câu (max length), d_{model} : kích thước embedding

- Ta học ba ma trận số $W_Q, W_K, W_V \in \mathbb{R}^{d_{model}, d_{model}}$ để $Q = XW_Q, K = XW_K, V = XW_V$

Nghĩa là từ mỗi token embedding, ta tạo ra 3 bản chiếu khác nhau:

- Q (Query): “tôi đang tìm thông tin gì?”
- K (Key): “tôi chứa thông tin gì?”
- V (Value): “thông tin thật của tôi là gì?”

- Sau đó, ta chia Q, K, V thành nhiều head. Transformer không chỉ dùng một “cách nhìn”, mà dùng nhiều head song song. Nghĩa là ta reshape Q, K, V thành $Q, K, V \in \mathbb{R}^{(B, h, L, d_k)}$.

Giả sử ta có: $h = 8, d_k = \frac{d_{model}}{h} = 64$. Tức là chia 512 chiều thành 8 lát (mỗi lát 64 chiều). Mỗi lát là 1 head riêng biệt, sẽ học một loại quan hệ khác nhau.

- Với mỗi i : $head_i = \text{Attention}(Q_i, K_i, V_i) = \text{softmax}\left(\frac{Q_i K_i^T}{\sqrt{d_k}}\right) V_i$

Trong đó:

- $Q_i K_i^T \in \mathbb{R}^{(B, L, L)}$ tính mức độ liên quan giữa các token (nhắc lại L là số token max)
- Chia $\sqrt{d_k}$ để tránh giá trị lớn làm softmax quá cực đoan
- Nhân với V_i tổng hợp thông tin từ các token liên quan. Từ đó, Kết quả: mỗi token đầu ra của $head_i$ là trung bình có trọng số của tất cả token khác, tùy mức độ quan tâm (attention).



- Khi đó, $\text{Concat}(head_1, head_2, \dots, head_h)$ mang toàn bộ góc nhìn từ h head khác nhau (ngữ pháp, ngữ nghĩa, cấu trúc câu...), nhân thêm với ma trận $W_O \in \mathbb{R}^{(d_{model}, d_{model})}$ quyết định trọng số để học từ mỗi góc nhìn.

Tóm lại,

$$\text{Output} = \text{Concat}(head_1, head_2, \dots, head_h) W_O$$

2.4 Residual Learning

Ý tưởng đằng sau Residual Learning: ta để mạng học phần sai lệch (residual) $\mathbf{F}$ giữa đầu ra $\mathbf{H}$ mong muốn và đầu vào x : $\mathbf{H}(x) = \mathbf{F}(x) + x$.

Khi backpropagation (lan truyền ngược), gradient cũng chia ra hai hướng: $\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \cdot \left(\frac{\partial F(x)}{\partial x} + I \right)$. Cấu trúc này giúp Gradient không bị tắc nghẽn khi mạng sâu; nếu block không giúp ích gì, nó “bỏ qua” $F(x) \approx 0$, và vẫn truyền gradient qua x bình thường.

2.5 Feed Forward

Mỗi token sau attention có ngữ cảnh tổng hợp. Sau khi token đã hiểu ngữ cảnh, FFN sẽ biến đổi nội bộ embedding đó, giúp mô hình học biểu diễn phi tuyến sâu hơn cho chính token đó. Không còn tương tác giữa các tokens ở đây — mỗi token xử lý độc lập.

Mỗi token (một vector embedding chiều d_{model}) được xử lý độc lập qua hai lớp linear (MLP nhỏ):

$$\text{FFN}(x) = \max(0, xW_1 + b_1) W_2 + b_2$$

3 Decoder block

3.1 Masked Multi-head Attention: Subsequent mask

Với self-attention, ta không muốn model thấy các từ đằng sau bước hiện tại. Nếu không, model sẽ học cheat và học cách nhìn trước tương lai. Nhưng vào thời điểm test, không có từ tương lai. Nếu không mask, mô hình sẽ dựa vào thông tin tương lai trong training, dẫn đến không khớp với cách sinh token step-by-step trong inference. Việc dùng subsequent mask đảm bảo mô hình luôn học phân phối auto-regressive đúng, chỉ dựa vào các token trước đó và hiện tại, từ đó cải thiện hiệu quả khi sinh câu mới.

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} + M \right) V$$

Trong đó:

$$M = \begin{bmatrix} 0 & -\infty & -\infty & \dots & -\infty \\ 0 & 0 & -\infty & \dots & -\infty \\ 0 & 0 & 0 & \dots & -\infty \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & 0 \end{bmatrix}$$

Các vị trí bị mask (tương lai) sẽ cộng với $-\infty \rightarrow \text{softmax}$ gần như 0 $\rightarrow$ không được nhìn.

3.2 Multi-head attention (trong Decoder)

Giả sử:

- batch_size = B
- sequence length (decoder input) = T
- sequence length (encoder output) = S
- hidden size of transformer = $d_{model} = D$
- number of heads = H



- dimension of each head = $d_k = D/H$
- Input:
 - Decoder input embeddings + positional encoding: $X_{dec} \in \mathbb{R}^{B \times T \times D}$
 - Encoder output: $X_{enc} \in \mathbb{R}^{B \times S \times D}$
 - Masked self-attention (decoder):

$$Q_{MSA} = X_{dec} W^Q, \quad K_{MSA} = X_{dec} W^K, \quad V_{MSA} = X_{dec} W^V$$

$$Q_{MSA}, K_{MSA}, V_{MSA} \in \mathbb{R}^{B \times H \times T \times d_k}$$

- Encoder-decoder attention:

$$Q_{EDA} = X_{dec} W^Q, \quad K_{EDA} = X_{enc} W^K, \quad V_{EDA} = X_{enc} W^V$$

$$Q_{EDA} \in \mathbb{R}^{B \times H \times T \times d_k}, \quad K_{EDA}, V_{EDA} \in \mathbb{R}^{B \times H \times S \times d_k}$$

- Decoder block:

- Masked self-attention output:

$$X_1 = \text{LayerNorm}(X_{input} + \text{MSA_output})$$

- Encoder-decoder attention output:

$$X_2 = \text{LayerNorm}(X_1 + \text{EDA_output})$$

- Feed-Forward layer:

$$X_{out} = \text{LayerNorm}(X_2 + \text{FFN}(X_2))$$

4 Training

$$X_{out}^{(B,S,D)} \xrightarrow{(D, \text{vocab_size})} X_{out}^{(B,S, \text{vocab_size})} \xrightarrow{\text{softmax}} X_{out}^{prob}$$

4.1 Loss

- Cross Entropy Loss: $\mathcal{L}_{BCE} = - \sum_{t=1}^{\text{decoder_length}} \sum_{i=1}^{\text{vocab_size}} y_i \log(\hat{y}_i)$
- KL Divergence Loss: $\mathcal{L}_{KL} = \sum_{t=1}^{\text{decoder_length}} \sum_{i=1}^{\text{vocab_size}} P_i \log \frac{P_i}{Q_i}$

Thông thường, khi áp dụng Cross-Entropy (CE) Loss, ta tính loss theo one-hot vector $y = [0, 0, \dots, 1, \dots, 0]$ trong đó 1 đại diện cho nhãn đúng. Tuy nhiên, đôi khi ta không muốn mô hình quá chắc chắn về lựa chọn này. Trong trường hợp đó, ta áp dụng label smoothing và có thể dùng teacher forcing với KL Divergence Loss:

- Chọn hệ số α .
- Áp dụng label smoothing:
 - Giá trị nhãn đúng: $1 - \alpha$
 - Giá trị nhãn sai: $\frac{\alpha}{\text{vocab_size} - 1}$
- Loss cuối cùng có thể được tính qua KL Divergence giữa phân phối “soft labels” (sau label smoothing) P và phân phối dự đoán của mô hình Q :

$$\mathcal{L}_{KL} = \sum_{t=1}^{\text{decoder_length}} \sum_{i=1}^{\text{vocab_size}} P_i \log \frac{P_i}{Q_i}$$

Trong đó P_i là nhãn đã được smoothed:

$$P_i = \begin{cases} 1 - \alpha, & \text{nếu } i = \text{nhãn đúng} \\ \frac{\alpha}{\text{vocab_size} - 1}, & \text{nếu } i \neq \text{nhãn đúng} \end{cases}$$



4.2 Dropout và Learning rate Warmup

Dropout là một kỹ thuật regularization được sử dụng trong mạng nơ-ron để giảm overfitting. Trong quá trình huấn luyện, một số neuron được “tắt” tạm thời với một xác suất nhất định, giúp mô hình không phụ thuộc quá mức vào bất kỳ neuron nào.

Learning Rate Warmup là phương pháp tăng dần learning rate từ một giá trị nhỏ ban đầu lên giá trị mục tiêu trong vài bước huấn luyện đầu tiên. Kỹ thuật này giúp mô hình ổn định hơn trong giai đoạn đầu của training, đặc biệt hữu ích với các mạng sâu và optimizer như Adam.

$$\text{lr} = \frac{1}{\sqrt{d_{\text{model}}}} \min \left(\frac{1}{\sqrt{t}}, \frac{t}{\text{warmup_steps}^{1.5}} \right)$$